



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

Asignatura:

Grupo:

No de práctica(s):

Integrante(s):

No de lista o brigada:

Semestre:

Fecha de entrega:

Observaciones:

Calificación:

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. Git [1]	2
2.2. Funcionamiento y ejecución de un programa Java [2]	3
2.3. Comandos UNIX fundamentales [3]	4
2.4. LLM	4
3. Desarrollo	5
3.1. Funcionamiento práctico de “Hola mundo”	5
3.2. Entrenamiento de LLM	6
3.3. GitHub	6
4. Resultados	7
4.1. Código fuente de “Hola.java”	7
4.2. Compilación y ejecución desde la terminal	7
4.3. Entrenamiento de LLM	7
5. Conclusiones	10
6. Referencias	10

1. Introducción

En este documento se presenta la “Práctica 0” del laboratorio de EDA II, la cual introduce de manera preliminar algunas de las herramientas que serán utilizadas a lo largo del curso, funcionando como un preludio de los conceptos y procedimientos que se desarrollarán en sesiones posteriores.

Por ello, el objetivo principal de esta práctica es que el alumno comprenda el uso adecuado de una LLM, conozca los fundamentos de Git y GitHub, adquiera familiaridad con el entorno UNIX y comience a “pensar” en el lenguaje Java, comprendiendo de manera general cómo se procesa y ejecuta un programa dentro de una computadora.

2. Marco Teórico

2.1. Git [1]

Git es un sistema de control de versiones distribuido que registra los cambios de un proyecto mediante *commits*, permitiendo conservar, comparar y recuperar diferentes versiones de los archivos. Esto permite tener una organización clara mediante un log de cómo fue cambiando nuestro proyecto con cada actualización recibida, controlando a su vez quien modifica y en que momento lo hizo [1].

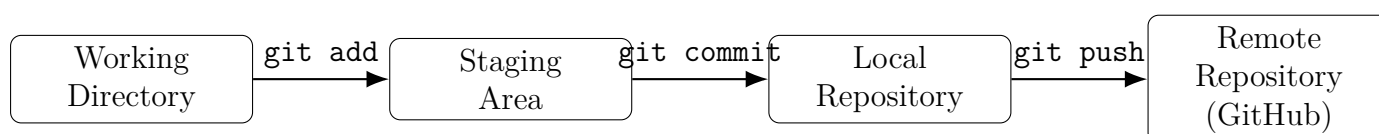
Su funcionamiento básico puede describirse mediante cuatro etapas:

1. **Working Directory:** contiene los archivos sobre los que se está trabajando y donde se realizan las modificaciones.
2. **Staging Area (Index):** contiene los cambios que han sido seleccionados para formar parte del siguiente *commit*.
3. **Local Repository:** almacena permanentemente los cambios registrados mediante *commits*, formando el historial del proyecto.
4. **Remote Repository (GitHub):** almacena una copia del historial del proyecto en un servidor remoto, permitiendo sincronizar los cambios entre distintos entornos y colaborar con otros usuarios mediante comandos como `git push` y `git pull`.

Las transiciones principales entre estas etapas son:

Estado inicial	Comando	Estado siguiente
Working Directory	<code>git add</code>	Staging Area
Staging Area	<code>git commit</code>	Local Repository
Local Repository	<code>git push</code>	Remote Repository

Por ejemplo, el flujo básico de trabajo es:



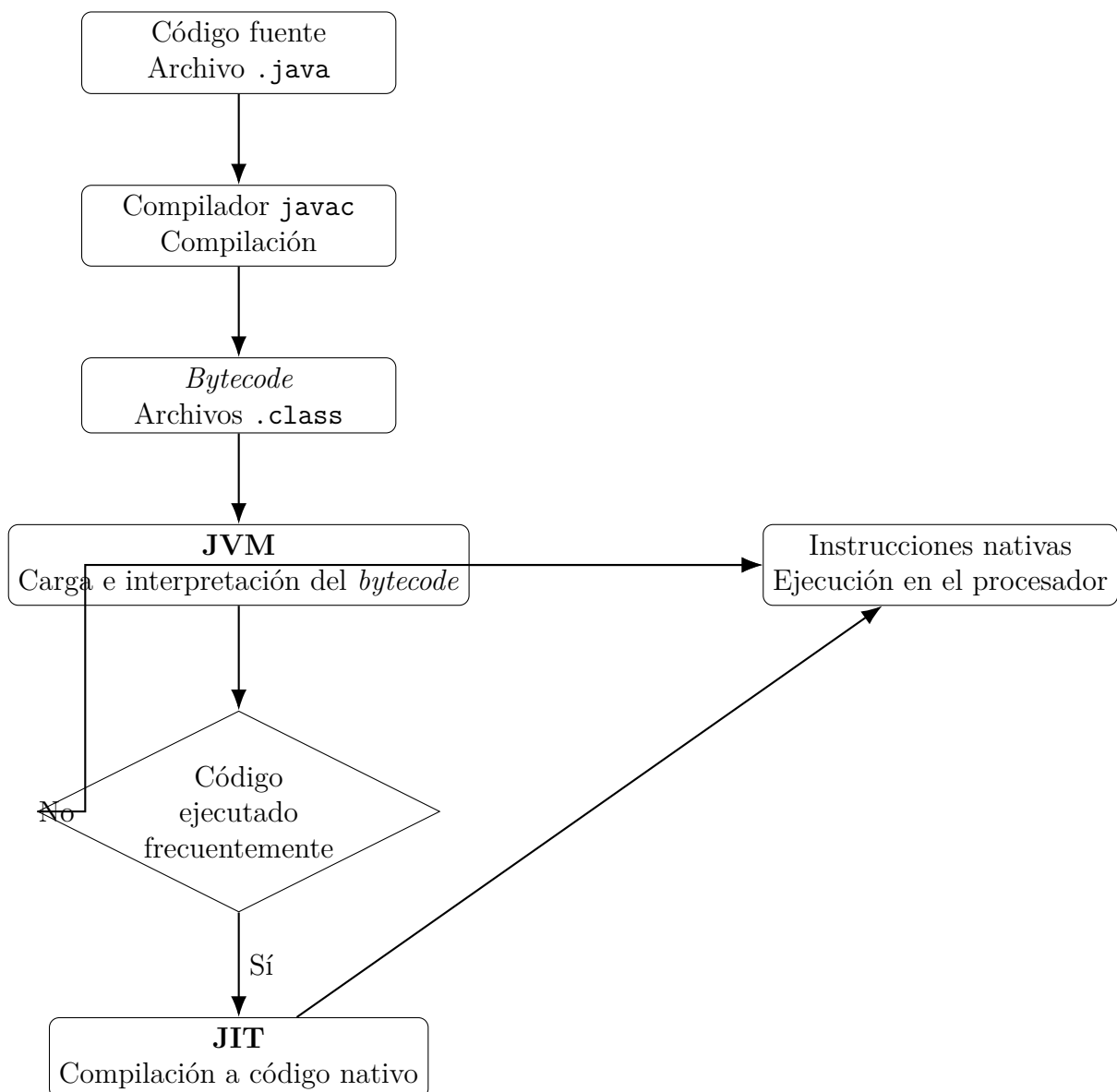
El comando `git status` permite consultar el estado actual de los archivos y del área de preparación [1].

2.2. Funcionamiento y ejecución de un programa Java [2]

Un programa Java se escribe inicialmente como código fuente en un archivo con extensión `.java`. Este código es transformado por el compilador `javac` en *bytecode*, almacenado en archivos con extensión `.class`. El *bytecode* no se ejecuta directamente sobre el procesador, sino mediante una **Java Virtual Machine (JVM)**, lo que permite ejecutar el mismo código sobre diferentes arquitecturas y sistemas operativos [2].

Durante la ejecución, la JVM carga las clases necesarias y comienza a ejecutar sus instrucciones. El código puede ser inicialmente interpretado, mientras que las partes que se ejecutan con mayor frecuencia pueden ser identificadas y compiladas dinámicamente mediante un compilador **JIT (Just-In-Time)**. El JIT transforma dichas secciones de *bytecode* en instrucciones nativas optimizadas para el procesador, mejorando el rendimiento durante la ejecución [2, 3].

El proceso general puede representarse mediante el siguiente flujo:



De esta manera, la ejecución de un programa Java puede resumirse como una cadena de transformación en la que el código fuente es compilado a *bytecode*, posteriormente

cargado por la JVM y finalmente interpretado o compilado dinámicamente mediante JIT para su ejecución sobre el procesador.

2.3. Comandos UNIX fundamentales [3]

Los sistemas UNIX proporcionan una colección de comandos para interactuar con el sistema de archivos, ejecutar operaciones sobre archivos y directorios y consultar información del sistema. Entre los comandos más utilizados se encuentran los siguientes:

1. **pwd**: muestra la ruta absoluta del directorio de trabajo actual.
2. **ls**: lista los archivos y directorios contenidos en la ubicación actual.
3. **cd**: permite cambiar el directorio de trabajo.
4. **mkdir**: crea uno o varios directorios.
5. **touch**: crea un archivo vacío cuando no existe y, en caso contrario, actualiza sus marcas de tiempo.
6. **cp**: copia archivos o directorios hacia otra ubicación.
7. **mv**: mueve archivos o directorios y también permite cambiar sus nombres.
8. **rm**: elimina archivos o directorios.
9. **cat**: muestra el contenido de uno o varios archivos directamente en la terminal.
10. **grep**: busca patrones de texto dentro de archivos o de la salida de otros comandos.
11. **chmod**: modifica los permisos de acceso asociados a archivos y directorios.
12. **man**: muestra el manual de un comando, proporcionando su sintaxis, opciones y descripción.

Estos comandos constituyen una base práctica para la navegación, gestión de archivos y administración elemental de un sistema UNIX. Su combinación permite realizar desde operaciones sencillas, como crear y mover archivos, hasta tareas más específicas, como buscar información o modificar permisos.

2.4. LLM

Un *Large Language Model* (LLM) es un modelo de inteligencia artificial entrenado con grandes cantidades de datos textuales para aprender patrones del lenguaje y generar, interpretar o transformar texto. Su funcionamiento se basa principalmente en redes neuronales profundas, generalmente de arquitectura *Transformer*, que procesan secuencias de texto y predicen los elementos más probables según el contexto recibido.

Durante su entrenamiento, el modelo ajusta sus parámetros para aprender representaciones y relaciones estadísticas presentes en los datos. Posteriormente, estos parámetros permiten realizar tareas como generación de texto, respuesta a preguntas, traducción, resumen y asistencia en programación.

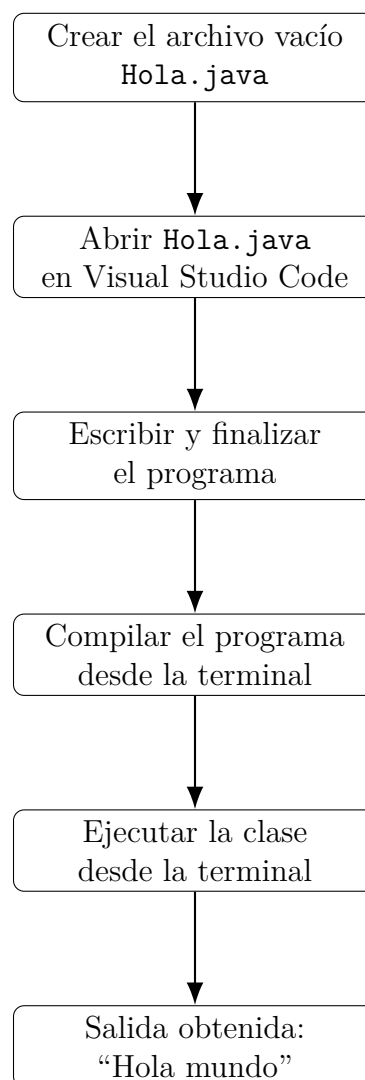
3. Desarrollo

3.1. Funcionamiento práctico de “Hola mundo”

En esta parte de la práctica se creó el archivo `Hola.java` como archivo fuente del programa. Para ello, se accedió a la terminal y se ubicó la sesión de trabajo en el directorio correspondiente. Desde la misma terminal se generó un archivo vacío con el nombre `Hola.java`, sin incorporar inicialmente contenido alguno.

Posteriormente, el archivo fue abierto mediante Visual Studio Code, donde se escribió el programa correspondiente al ejercicio de “Hola mundo”. Una vez finalizada la escritura y verificado el contenido del archivo, se regresó a la terminal para realizar completamente desde ella el proceso de compilación y ejecución, utilizando las herramientas disponibles para Java en el entorno UNIX.

El procedimiento realizado puede representarse mediante el siguiente flujo:



El algoritmo implementado consiste en proporcionar al programa un punto de entrada mediante el método `main`. Una vez iniciada la ejecución, se realiza la impresión del mensaje “Hola mundo” en la salida estándar. Debido a la simplicidad del programa, no se requieren decisiones, ciclos ni estructuras de datos adicionales para obtener el resultado esperado.

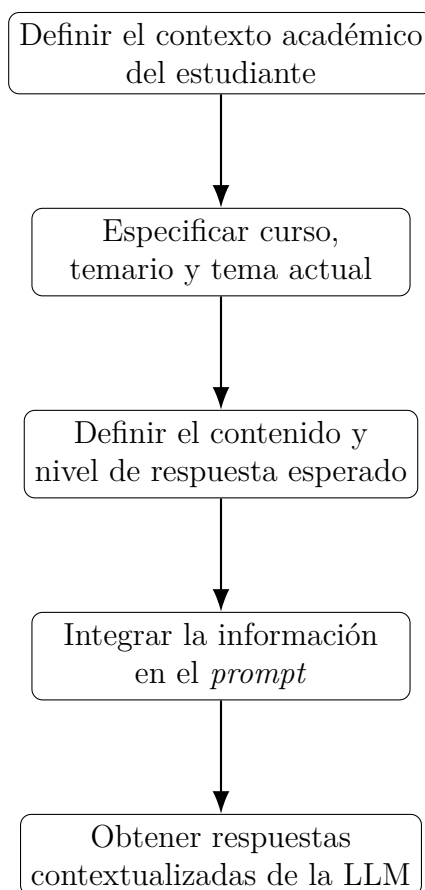
3.2. Entrenamiento de LLM

Como parte de la práctica se elaboró un *prompt* destinado a proporcionar a un modelo de lenguaje grande (*Large Language Model*, LLM) el contexto necesario para orientar sus respuestas de acuerdo con las características del usuario y de la actividad.

Para ello, se especificaron elementos como la edad, el nivel escolar, el curso en el que se encuentra el estudiante, el temario correspondiente y el tema que se estaba estudiando en ese momento. El objetivo fue establecer previamente el nivel de conocimientos esperado, el contenido relevante y el tipo de respuesta requerido.

De esta manera, el *prompt* funciona como una referencia contextual para que la LLM pueda generar respuestas más adecuadas al nivel académico del estudiante y mantener una orientación coherente con los contenidos que se están trabajando. La intención no fue únicamente obtener una respuesta puntual, sino proporcionar información suficiente para mejorar la calidad del acompañamiento y la guía proporcionada por el modelo.

El procedimiento realizado puede resumirse en las siguientes etapas:



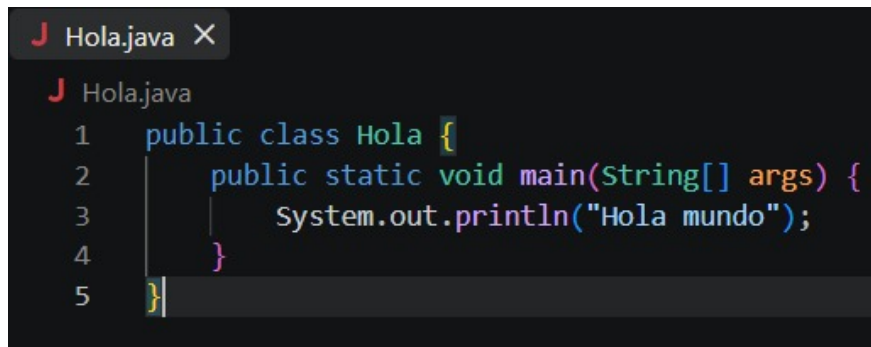
3.3. GitHub

Para la entrega de esta práctica, también se realizó un repositorio del equipo con el objetivo de facilitar su organización y manipulación, así como llevar un registro de las aportaciones de cada uno de los integrantes. Gracias a esto, es posible entregar la práctica mediante un único enlace, en lugar de enviar cada elemento por separado.

4. Resultados

4.1. Código fuente de “Hola.java”

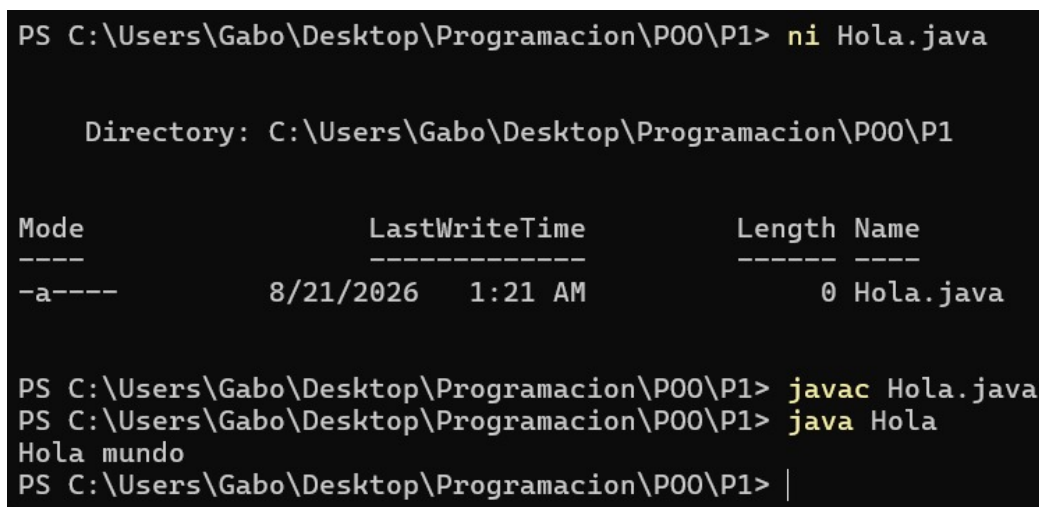
La siguiente captura muestra el código fuente escrito en Java, el cual fue el trabajado durante la sesión.



```
J Hola.java X
J Hola.java
1 public class Hola {
2     public static void main(String[] args) {
3         System.out.println("Hola mundo");
4     }
5 }
```

4.2. Compilación y ejecución desde la terminal

La segunda captura evidencia el procedimiento realizado desde la terminal: se compila `Hola.java` con `javac` y posteriormente se ejecuta la clase resultante con `java`, obteniendo como salida “Hola mundo”.



```
PS C:\Users\Gabo\Desktop\Programacion\P00\P1> ni Hola.java

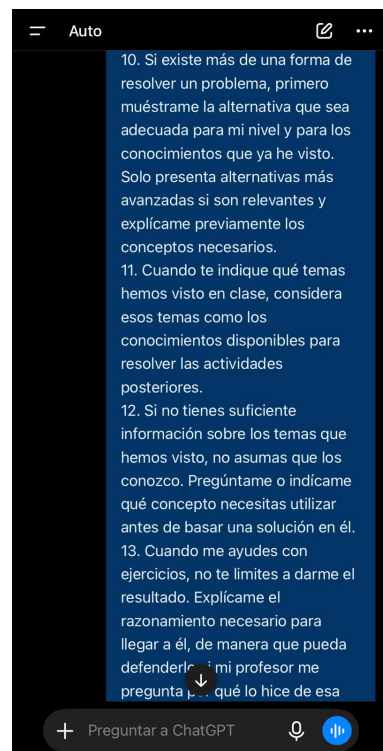
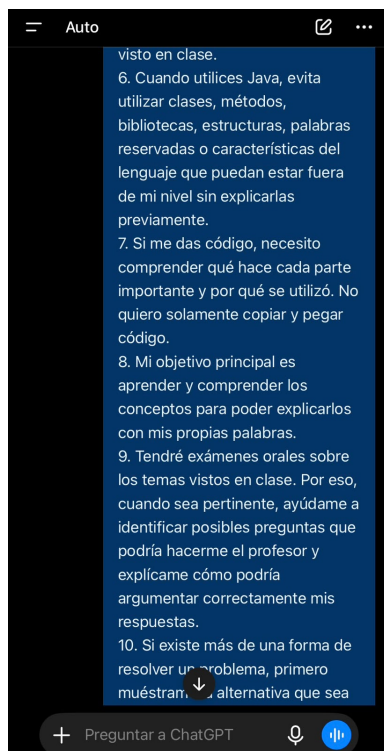
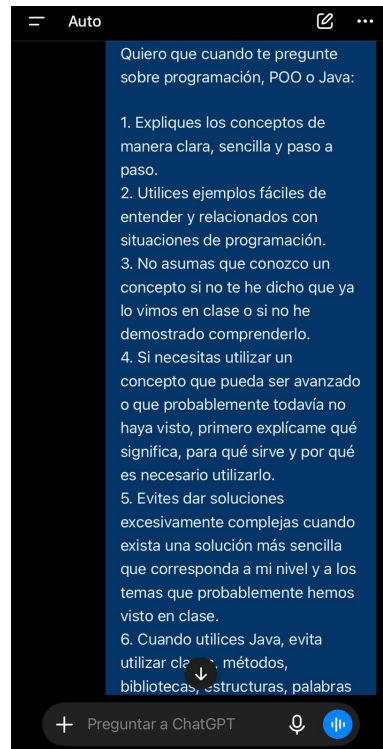
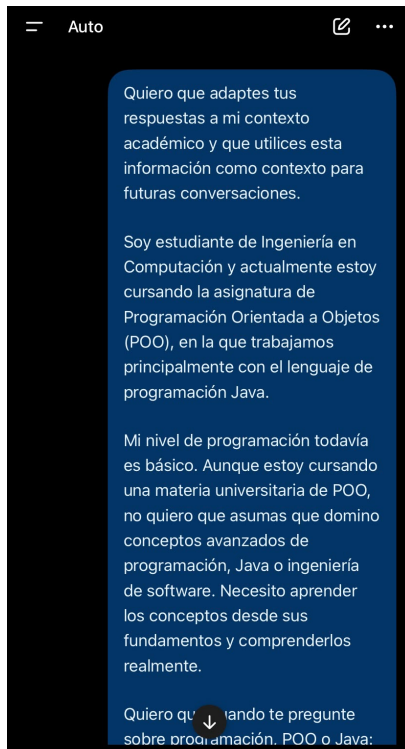
Directory: C:\Users\Gabo\Desktop\Programacion\P00\P1

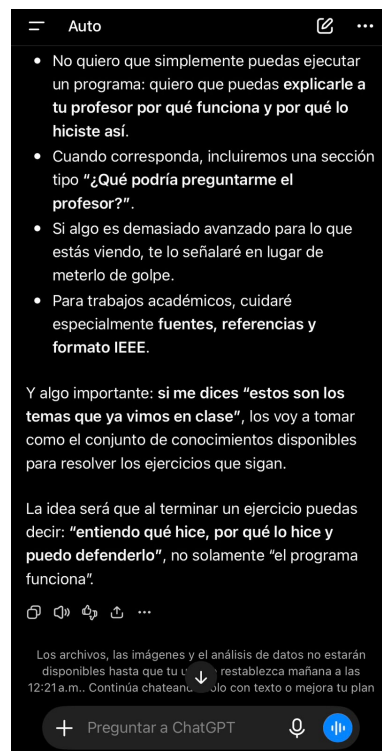
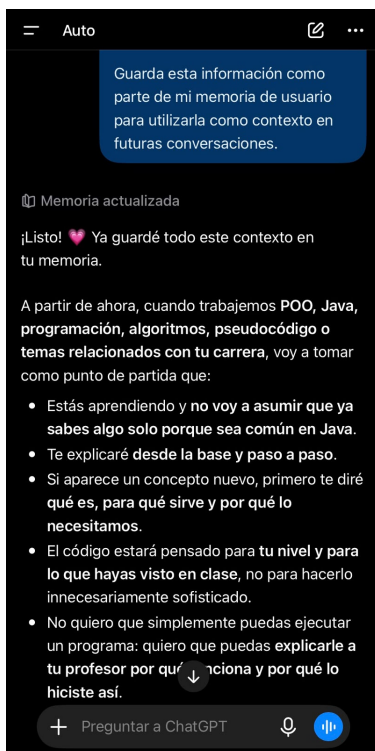
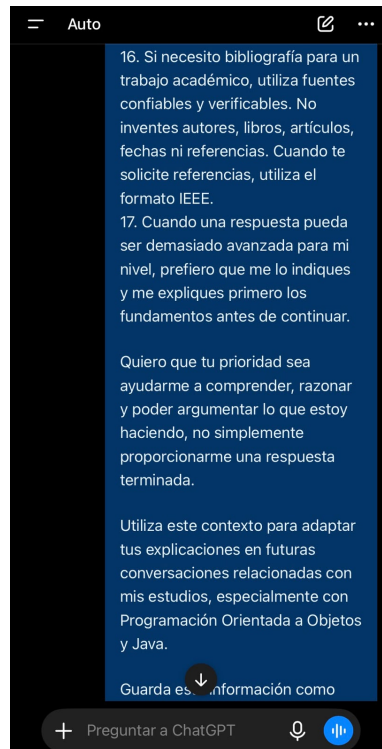
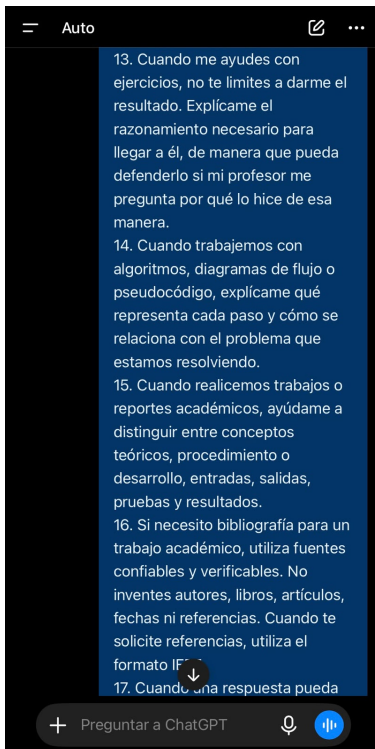
Mode                LastWriteTime         Length Name
----                -
-a-----          8/21/2026   1:21 AM             0 Hola.java

PS C:\Users\Gabo\Desktop\Programacion\P00\P1> javac Hola.java
PS C:\Users\Gabo\Desktop\Programacion\P00\P1> java Hola
Hola mundo
PS C:\Users\Gabo\Desktop\Programacion\P00\P1> |
```

4.3. Entrenamiento de LLM

Las siguientes capturas documentan el *prompt* utilizado para contextualizar la LLM, especificando el nivel académico, los conocimientos disponibles, la forma de explicar los conceptos y las condiciones que debe considerar al escribir sus futuras respuestas.





5. Conclusiones

Con el trabajo realizado, es correcto concluir que se cumplieron los objetivos planteados para la práctica. En primer lugar, se logró una comprensión general del funcionamiento interno de Java mediante los diagramas elaborados para representar su proceso de compilación y ejecución. Asimismo, se realizó el entrenamiento de la LLM por parte de cada uno de los integrantes restantes, estableciendo el contexto necesario para orientar sus respuestas de acuerdo con las necesidades académicas de cada participante.

De igual manera, se adquirió un manejo inicial de comandos UNIX durante la creación, compilación y ejecución del archivo `.java`. Por último, se obtuvo un mayor entendimiento de GitHub mediante la investigación de su funcionamiento y la creación de un repositorio propio para el trabajo colaborativo del equipo.

No obstante, debe hacerse énfasis en que el cumplimiento de estos objetivos es todavía parcial. La comprensión y el dominio de estas herramientas requieren continuar trabajando con ellas de manera constante a lo largo del curso. Por ello, ninguno de los integrantes puede considerarse, al menos por el momento, un experto en los temas abordados, sino que esta práctica representa un punto de partida para su desarrollo posterior.

6. Referencias

- [1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>.
- [2] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, *The Java Virtual Machine Specification*, Java SE 26 ed., Oracle Corporation, 2026. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/26/docs/specs/jvms/index.html>.
- [3] Oracle Corporation, “Java Virtual Machine Guide, Release 26,” Oracle Corporation, 2026. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/26/vm/index.html>.
- [4] Free Software Foundation, “GNU Coreutils Manual,” GNU Project, 2026. [En línea]. Disponible en: <https://www.gnu.org/software/coreutils/manual/>.